Tarea 5 – Modelos generativos

Traducción de texto a emojis con un Transformer encoder-decoder

Análisis completo del notebook: qué pedía la tarea, el enfoque, el código de cada celda, lo que se hizo y los resultados obtenidos

Diplomado en Machine Learning Avanzado

Módulo 5 – Unidad 5 · Deep Learning

Universidad Nacional de Colombia – Facultad de Ingeniería

Notebook: `v2 M5U5 Tarea 5.ipynb`

Documento elaborado el 7 de noviembre de 2026

Contenido

1. Qué pedía la tarea
2. El enfoque: un Transformer encoder-decoder
3. Conceptos que se repiten en todo el notebook
4. Flujo general del notebook
5. Explicación celda por celda
    a. Configuración y calificador
    b. Carga de datos
    c. Punto 1 – Preprocesamiento
    d. Punto 2 – Vectorización
    e. Punto 3 – Codificador
    f. Punto 4 – Decodificador
    g. Punto 5 – Construir el modelo
    h. Punto 6 – Entrenamiento
    i. Generación de ejemplo
    j. Evaluación y envío
6. El hilo conductor: una frase de principio a fin
7. Resultados y análisis
8. Lo que hubo que corregir para obtener el 100 %
9. Conclusiones

1. Qué pedía la tarea

La tarea plantea un problema de **generación de secuencias**: dado un texto en inglés, producir una secuencia de emojis que capture su significado y su contexto emocional. El enunciado aclara que esto va más allá de traducir palabra por palabra: el modelo debe comprender el contexto semántico, identificar conceptos clave y emociones implícitas, generar secuencias coherentes de emojis y mantener la fluidez en la representación visual.

Es, en esencia, el mismo tipo de problema que la traducción automática entre idiomas (*sequence-to-sequence*, o seq2seq): entra una secuencia de longitud variable y sale otra secuencia, de otro "idioma" y de otra longitud.

Los datos

El conjunto de datos es un archivo JSON con unas 10 000 parejas. Cada pareja tiene una frase en inglés (en su mayoría intervenciones del Parlamento Europeo) y su representación en emojis. Ejemplos reales de las primeras filas:

Texto en inglés	Emojis
The ratification and implementation of the updated ILO conventions vote	✅🗳️
Plus we agreed to delete point a	❌
Parliament agreed to accept the oral amendment	✅💬

Los seis puntos calificados

El notebook está organizado en seis funciones que el estudiante debe completar. Cada una tiene su propia celda de prueba local y una o dos celdas `grader.run_test(...)` que la envían al calificador automático de UNCode.

Punto	Función	Qué debe hacer
1	`preprocess`	Limpiar el texto en inglés y añadir tokens de inicio y fin a ambas columnas.
2	`vectorize`	Convertir texto y emojis en secuencias de números de longitud fija.
3	`get_EncoderLayer`	Implementar el codificador de un Transformer como capa personalizada de Keras.
4	`get_DecoderLayer`	Implementar el decodificador de un Transformer como capa personalizada.
5	`get_model`	Unir codificador y decodificador en un modelo completo.
6	`train_model`	Compilar y entrenar el modelo, desfasando la salida un token.

Cómo califica el grader

El calificador no se limita a revisar que el código "funcione". Ejecuta cada función con sus propias entradas y compara el resultado con una solución de referencia. En algunos tests compara **formas** (dimensiones de las matrices), en otros compara **valores numéricos exactos** y en otros inspecciona

la **estructura interna** de las capas: qué atributos existen, de qué tipo son y con qué parámetros fueron creados. Este detalle fue decisivo, como se verá en la sección 8.

2. El enfoque: un Transformer encoder-decoder

La arquitectura elegida (y exigida por el enunciado) es el Transformer, propuesto en el artículo *Attention Is All You Need* (Vaswani et al., 2017). Tiene dos mitades:

> **Codificador (encoder).** Lee la frase completa en inglés y produce, para cada palabra, un vector numérico que ya incorpora información de las demás palabras de la frase. Es la parte que "entiende" la entrada.
>
> **Decodificador (decoder).** Genera los emojis uno por uno. En cada paso mira lo que ya generó (para mantener coherencia) y consulta la salida del codificador (para no perder de vista el significado de la frase original). Es la parte que "escribe" la salida.

El mecanismo que hace posible todo esto es la **atención**: en lugar de procesar las palabras en orden, como hacían las redes recurrentes, cada palabra puede "mirar" directamente a cualquier otra y decidir cuánto peso darle. Eso permite captar relaciones a larga distancia y procesar toda la secuencia en paralelo.

En esta tarea el modelo es deliberadamente pequeño: un solo bloque de encoder, un solo bloque de decoder, embeddings de 128 dimensiones, 2 cabezas de atención y una red interna de 32 neuronas. Aun así suma 2,3 millones de parámetros.

3. Conceptos que se repiten en todo el notebook

Token

La unidad mínima que procesa el modelo. En el texto en inglés, cada palabra es un token. En la columna de emojis, como se verá, cada cadena completa de emojis termina siendo un solo token.

Tokens especiales [BOS] y [EOS]

`[BOS]` (*Beginning Of Sequence*) marca el inicio y `[EOS]` (*End Of Sequence*) marca el fin. No son palabras reales. El decodificador necesita `[BOS]` como punto de partida para generar el primer emoji, y aprende a producir `[EOS]` para indicar que terminó. Sin ellos, no sabría ni cómo empezar ni cuándo parar.

Vectorización y relleno (padding)

Las redes neuronales solo procesan números. Cada token se reemplaza por un identificador entero y todas las secuencias se llevan a la misma longitud rellenando con ceros. En este notebook la longitud es 26.

Embedding

Una tabla que convierte cada identificador entero en un vector denso (aquí de 128 números). Al inicio los vectores son aleatorios; con el entrenamiento se ajustan para que tokens con significado parecido

queden cerca en ese espacio.

Embedding de posición

La atención, por sí sola, no sabe en qué orden van los tokens. Se suma un segundo embedding que codifica la posición (0, 1, 2, …) para que el modelo recupere la noción de orden.

Atención multi-cabeza (MultiHeadAttention)

Cada token calcula cuánto debe fijarse en cada uno de los demás y construye su nueva representación como una mezcla ponderada. Con varias "cabezas" (aquí 2) cada una puede especializarse en un tipo de relación distinto. Cuando una secuencia se mira a sí misma, `att(x, x)`, se llama **auto-atención**; cuando mira a otra secuencia, `att(x, contexto)`, se llama **atención cruzada**.

Máscara causal

En el decodificador, la posición *i* solo puede mirar las posiciones ≤ *i*. Así, durante el entrenamiento, el modelo no puede "ver" el emoji que debe predecir. Es lo que hace que el modelo sea generativo de izquierda a derecha.

Conexión residual + normalización de capa

Tras cada sub-bloque se calcula `norm(x + salida)`. Sumar la entrada original evita que la información se pierda y facilita el flujo del gradiente; la normalización estabiliza la escala de los valores.

Dropout

Apaga neuronas al azar durante el entrenamiento para que el modelo no dependa de ninguna en particular y generalice mejor. En inferencia no actúa.

Teacher forcing y desfase de un token

Durante el entrenamiento, el decodificador recibe la secuencia correcta de emojis desplazada: ve `[BOS] a b` y debe predecir `a b [EOS]`. En cada posición aprende a predecir el **siguiente** token dado lo anterior.

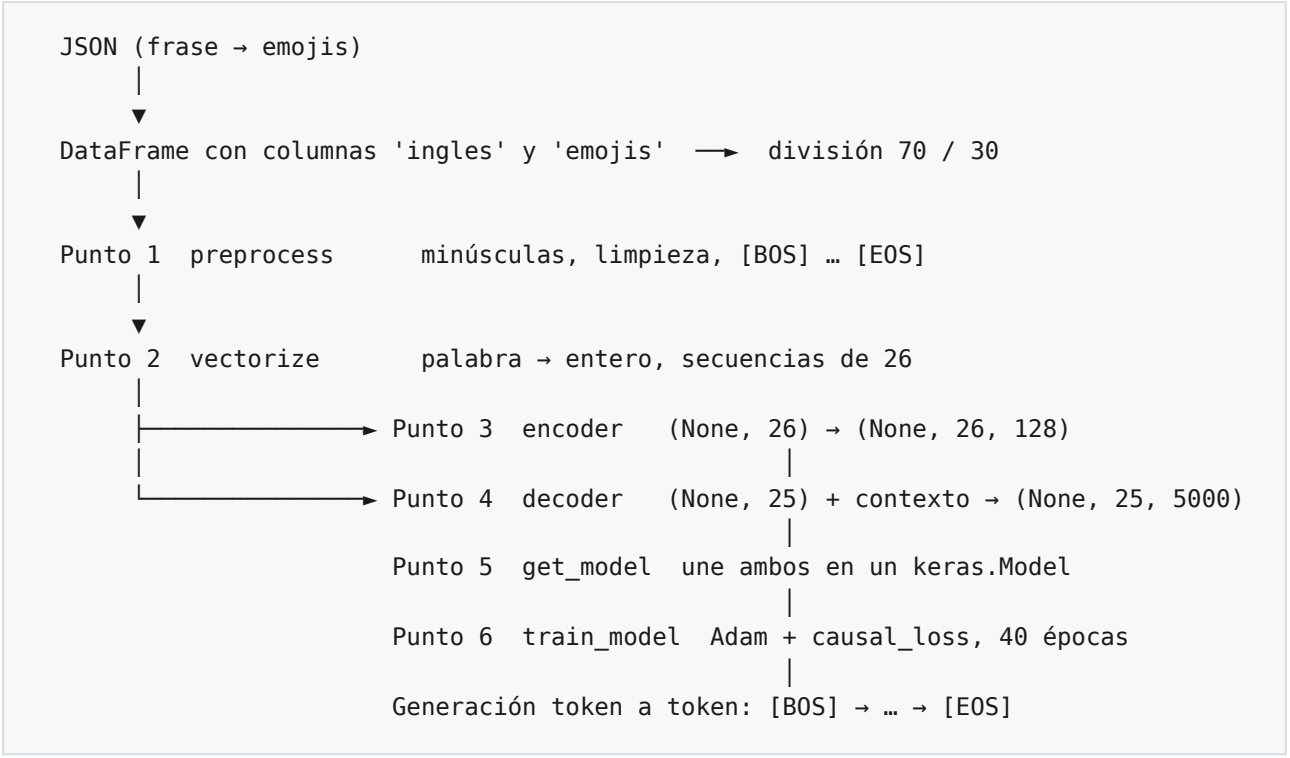
Decodificación autorregresiva

En el momento de generar de verdad, el modelo arranca con `[BOS]`, predice un token, lo añade a la entrada, vuelve a predecir, y así hasta producir `[EOS]`.

Conjuntos de entrenamiento y prueba

El modelo ajusta sus parámetros con el conjunto de entrenamiento y se mide con el de prueba, que son frases que nunca vio. Si solo se midiera con lo que entrenó, un resultado perfecto no distinguiría entre haber aprendido el patrón y haber memorizado las respuestas. Las métricas `val_loss` y `val_accuracy` son las que informan si el modelo generaliza.

4. Flujo general del notebook



5. Explicación celda por celda

En esta sección se recorre el notebook en el orden exacto en que aparece. Para cada celda se indica qué hace, se muestra el código completo tal como quedó en el notebook, se explica línea por línea y se muestra la salida que se obtuvo al ejecutarla.

5a. Configuración y calificador

CELDA 1 Instalación del calificador

```
CÓDIGO
%%capture
!pip install rlxcrypt
!wget --no-cache -O session.pye -q
https://raw.githubusercontent.com/JuezUN/INGInious/master/external%20libs/session.pye
```

- `%%capture` es un comando mágico de Jupyter que oculta toda la salida de la celda.
- El signo `!` ejecuta un comando de terminal desde el notebook.
- `rlxcrypt` es la librería que descifra el módulo del calificador, que viene encriptado para que no se pueda leer cómo califica.
- `wget` descarga ese módulo con el nombre `session.pye`.

Salida: ninguna (está capturada). **Qué se hizo:** nada, la celda venía lista.

CELDA 2 Inicio de sesión en UNCode

```
CÓDIGO
import rlxcrypt
import session

grader = session.LoginSequence("DL-GroupMLDS-5-2026-2@8c487bd7-4c95-4b75-88d5-
9e7093aae729")
```

Crea el objeto `grader`, vinculado al curso y a la tarea mediante el identificador entre comillas. Al ejecutarse pide el usuario y la contraseña de UNCode. Todas las celdas que empiezan con `grader.` dependen de esta. **Qué se hizo:** nada; las credenciales las introduce la estudiante.

CELDA 3 Librerías y semillas

CÓDIGO

```
import keras
import numpy as np
import matplotlib.pyplot as plt
import os, random
import pandas as pd
import re
import json

%matplotlib inline
plt.style.use("ggplot")
# Seleccionamos las semillas para efectos de reproducibilidad
random.seed(0)
keras.utils.set_random_seed(0)
np.random.seed(0)
```

- `keras`: construcción y entrenamiento de redes neuronales. `numpy`: arreglos numéricos. `pandas`: tablas. `re`: expresiones regulares para limpiar texto. `json`: lectura del archivo de datos.
- Las tres **semillas** fijan los generadores de números aleatorios de Python, Keras y NumPy. Los pesos de una red se inicializan al azar; con semilla fija salen siempre iguales. Como el grader compara valores numéricos exactos en varios tests, sin semillas nunca coincidirían.

CELDA 4 Versiones

CÓDIGO

```
# Versiones de las librerías usadas.
!python --version
print('NumPy', np.__version__)
print('keras', keras.__version__)
```

SALIDA

```
Python 3.13.16
NumPy 2.4.6
keras 3.13.2
```

Informativa. Las versiones del entorno son más recientes que las indicadas en el enunciado (Python 3.11, NumPy 2.0, Keras 3.8), pero todos los tests pasaron igual.

5b. Carga de datos

CELDA 5 Descarga del archivo

CÓDIGO

```
!pip install gdown
!gdown https://drive.google.com/uc?id=1HjGoRLLnLdEWjB1peXGFQXNGvbPRaAlj
```

```
SALIDA
Downloading...
From: https://drive.google.com/uc?id=1HjGoRLLnLdEWjB1peXGFQXNGvbPRaAlj
To: /content/emoji_translations.json
100% 1.09M/1.09M [00:00<00:00, 12.0MB/s]
```

`gdown` descarga archivos de Google Drive a partir de su identificador. Deja `emoji_translations.json` (1,09 MB) en la carpeta de trabajo de Colab.

CELDA 6 Lectura y división en entrenamiento y prueba

```
CÓDIGO
with open("/content/emoji_translations.json", 'r', encoding='utf-8') as f:
    data_dict = json.load(f)

data = pd.DataFrame(data_dict.items(), columns=['ingles', 'emojis'])
data = data.dropna()

train_texts = data.loc[:int(len(data)*0.7)]
test_texts = data.loc[int(len(data)*0.7):]

print(f'Numero de textos de entrenamiento: {len(train_texts)}')
print(f'Numero de textos de prueba: {len(test_texts)}')
```

```
SALIDA
Numero de textos de entrenamiento: 6605
Numero de textos de prueba: 3129
```

- `with open(..., 'r', encoding='utf-8') as f`: abre el archivo en modo lectura. UTF-8 es indispensable porque los emojis son caracteres Unicode. El bloque `with` cierra el archivo automáticamente al terminar.
- `json.load(f)`: convierte el contenido en un diccionario de Python `{frase: emojis}`.
- `data_dict.items()` entrega los pares (clave, valor); `pd.DataFrame(...)` convierte cada par en una fila y `columns=[...]` nombra las columnas `ingles` y `emojis`.
- `dropna()` elimina las filas con valores vacíos. No renumera el índice, así que pueden quedar huecos.
- `data.loc[:N]` toma desde el inicio hasta la etiqueta N **incluida**; `data.loc[N:]` desde N hasta el final. Como `.loc` trabaja con etiquetas y los huecos de `dropna()` desplazan la cuenta, la división real es 67,9 % / 32,1 % y la fila N queda en ambos conjuntos. No es un error que deba corregirse: el notebook venía así y el grader espera exactamente 6605 filas de entrenamiento.
- Los `print` usan *f-strings*: lo que va entre llaves se evalúa y se inserta en el texto.

CELDA 7 Primeras filas

```
CÓDIGO
train_texts.head()
```

Muestra las cinco primeras filas de la tabla para ver el formato de los datos. La fila 0, que servirá como ejemplo guía en la sección 6, es *“The ratification and implementation of the updated ILO conventions vote”* → ✅🗳️.

5c. Punto 1 – Preprocesamiento

> **Qué se pedía.** Para el texto: convertir a minúsculas, eliminar caracteres especiales (dejar solo letras y espacios), eliminar espacios repetidos y añadir `[BOS]` y `[EOS]`. Para los emojis: únicamente añadir `[BOS]` y `[EOS]`.

CELDA 8 Función calificada `preprocess`

CÓDIGO

```
# FUNCIÓN CALIFICADA preprocess
def preprocess(data):
    def preprocess_text(text):
        text = text.lower()                          # minúsculas
        text = re.sub(r'[^a-z\s]', '', text)         # solo letras y espacios
        text = re.sub(r'\s+', ' ', text).strip()     # espacios repetidos
        return '[BOS] ' + text + ' [EOS]'
    def preprocess_emoji(emoji):
        return '[BOS] ' + emoji + ' [EOS]'

    text_out = data.copy()
    text_out['ingles'] = text_out['ingles'].map(preprocess_text)
    text_out['emojis'] = text_out['emojis'].map(preprocess_emoji)
    return text_out
```

Línea por línea:

- `text.lower()`: “Hello” y “hello” pasan a ser la misma palabra; reduce el vocabulario.
- `re.sub(r'[^a-z\s]', '', text)`: la expresión regular `[^a-z\s]` significa “cualquier carácter que **no** sea una letra minúscula ni un espacio en blanco”. Se reemplaza por la cadena vacía, es decir, se borra. Así desaparecen números, puntuación, paréntesis y apóstrofes.
- `re.sub(r'\s+', ' ', text)`: `\s+` es “uno o más espacios en blanco seguidos”; se sustituyen por un solo espacio. `.strip()` elimina los espacios al inicio y al final.
- `'[BOS] ' + text + ' [EOS]'`: añade las marcas de inicio y fin, separadas por un espacio para que el vectorizador las reconozca como tokens independientes.
- `preprocess_emoji` solo añade las marcas; no modifica los emojis.
- `data.copy()`: se trabaja sobre una copia para no alterar el DataFrame original.
- `.map(función)`: aplica la función a cada celda de la columna y devuelve la columna transformada.

CELDA 9 Celda de prueba

CÓDIGO

```
# TEST CELL
train_texts_preprocess = preprocess(train_texts.copy())
test_texts_preprocess = preprocess(test_texts.copy())

print('Texto en inglés')
print(train_texts_preprocess.iloc[0]['ingles'])
print('Emojis')
print(train_texts_preprocess.iloc[0]['emojis'])
```

```
SALIDA
Texto en inglés
[BOS] the ratification and implementation of the updated ilo conventions vote [EOS]
Emojis
[BOS] ✅🗳️ [EOS]
```

Se aplica la función a ambos conjuntos y se imprime la fila 0. “The” pasó a “the”, “ILO” a “ilo” y la frase quedó delimitada. Los emojis conservan su forma original entre las marcas.

CELDAS 10–11 `grader.run_test("Test 1_1")` y `("Test 1_2")`

El grader ejecuta `preprocess` con sus propios textos de prueba (frases con mayúsculas, puntuación y paréntesis, como *“I'm so happy (today)”* o *“Let's eat some pizza!!!”*) y compara el DataFrame resultante con el esperado. **Resultado: 100 y 100.**

> **Qué hubo que corregir.** La “salida esperada” impresa en el notebook muestra los emojis separados por espacios (✅ 🗳️), y la primera versión los separaba carácter por carácter. Con eso el grader daba 80/100 con el mensaje “El resultado de text_out no es el esperado”. Probando cuatro variantes contra el grader se comprobó que espera los emojis **sin separar**, exactamente como dice la instrucción escrita. Esta decisión tiene una consecuencia importante en la vectorización, que se explica en el siguiente punto.

5d. Punto 2 – Vectorización

> **Qué se pedía.** Crear y ajustar una capa de vectorización para el texto y otra para los emojis. Entradas: el DataFrame preprocesado, la longitud máxima de secuencia (`seq_len`) y el tamaño máximo del vocabulario (`vocab_size`).

CELDA 12 **Función calificada** `vectorize`

```
CÓDIGO
# FUNCIÓN CALIFICADA vectorize
def vectorize(X, seq_len, vocab_size):
    en_vectorizer = keras.layers.TextVectorization(
        max_tokens=vocab_size, output_mode='int',
        output_sequence_length=seq_len + 1, standardize=None)
    em_vectorizer = keras.layers.TextVectorization(
        max_tokens=vocab_size, output_mode='int',
        output_sequence_length=seq_len + 1, standardize=None)
    en_vectorizer.adapt(X['ingles'].values)
    em_vectorizer.adapt(X['emojis'].values)
    return en_vectorizer, em_vectorizer
```

Parámetro por parámetro:

- `TextVectorization` es la capa de Keras que convierte texto en números. Por defecto separa los tokens por espacios.
- `max_tokens=vocab_size` (5000): tamaño máximo del vocabulario. Las palabras que no caben entre las 5000 más frecuentes se marcan como `[UNK]` (desconocida).
- `output_mode='int'` : cada token se convierte en un entero.

- `output_sequence_length=seq_len + 1` (26): toda secuencia se rellena con ceros o se recorta hasta medir exactamente 26. Así todas las frases caben en una misma matriz. Es `seq_len + 1` porque el entrenamiento quita un token por cada extremo y necesita que queden 25.
- `standardize=None`: por defecto la capa pasa a minúsculas y **elimina la puntuación**, lo que destruiría los corchetes de `[BOS]` y `[EOS]`. Como el texto ya viene limpio del punto 1, se desactiva.
- `.adapt(datos)`: recorre los textos de entrenamiento, cuenta los tokens y construye el vocabulario ordenado por frecuencia. Solo se adapta con **entrenamiento**, nunca con prueba, para no contaminar la evaluación.

CELDA 13 Celda de prueba

CÓDIGO

```
# TEST_CELL
en_vectorizer, emoji_vectorizer = vectorize(train_texts_preprocess, 25, 5000)
train_en_vec, train_emoji_vec = en_vectorizer(train_texts_preprocess['ingles']).numpy(),
emoji_vectorizer(train_texts_preprocess['emojis']).numpy()
test_en_vec, test_emoji_vec = en_vectorizer(test_texts_preprocess['ingles']).numpy(),
emoji_vectorizer(test_texts_preprocess['emojis']).numpy()

print(f'Tamaño de las entradas: {train_en_vec.shape}')
print(f'Tamaño de las salidas: {train_emoji_vec.shape}')
```

SALIDA

```
Tamaño de las entradas: (6605, 26)
Tamaño de las salidas: (6605, 26)
```

Se crean los vectorizadores y se transforman los cuatro conjuntos (inglés y emojis, entrenamiento y prueba). El resultado son matrices de 6605 filas × 26 columnas: una fila por frase, una columna por posición. `.numpy()` convierte el tensor de Keras en un arreglo de NumPy.

El vocabulario resultante asigna los primeros identificadores de forma fija: `0` = relleno, `1` = `[UNK]`, `2` = `[EOS]`, `3` = `[BOS]`. El resto se asigna por frecuencia. Estos números explican los valores "mágicos" `3` y `2` que aparecen en la celda de generación.

> **Consecuencia de no separar los emojis.** Al inspeccionar el vocabulario de emojis, sus primeros tokens fueron:
>
> ```
> ['', '[UNK]', '[EOS]', '[BOS]', '🤔❓', '✅👍', '❓🤔', '✅', '🤔', '🤔📝❓']
> ```
>
> Como la capa separa por espacios y los emojis no llevan espacios entre sí, **cada cadena completa de emojis es un único token** (🤔❓ es un token, no dos). La secuencia de emojis de la fila 0 queda entonces con solo tres tokens reales: `[3, id(✅🗳️), 2, 0, 0, …]`. Esto convierte la tarea, en la práctica, en elegir una cadena completa entre miles de combinaciones, lo que la hace más difícil para el modelo. Se retoma en el análisis de resultados.

CELDAS 14–15 Test 2_1 y 2_2

Resultado: 100 y 100, desde el primer intento.

5e. Punto 3 – Codificador

> **Qué se pedía.** Una capa personalizada de Keras (`keras.layers.Layer`) que represente el codificador de un Transformer. Parámetros: tamaño del vocabulario, longitud máxima, dimensión del embedding, número de cabezas, dimensión de la red feed-forward y tasa de dropout.

CELDA 16 Función calificada `get_EncoderLayer`

CÓDIGO

```
# FUNCIÓN CALIFICADA get_EncoderLayer
def get_EncoderLayer(vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate):

    class encoder(keras.layers.Layer):
        def __init__(self, vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate=0.1):
            super().__init__()
            self.token_emb = keras.layers.Embedding(input_dim=vocab_size,
output_dim=embed_dim)
            self.pos_emb = keras.layers.Embedding(input_dim=maxlen + 1,
output_dim=embed_dim)
            self.self_att = keras.layers.MultiHeadAttention(num_heads=num_heads,
key_dim=embed_dim, dropout=rate)
            self.self_layernorm = keras.layers.LayerNormalization()
            self.self_dropout = keras.layers.Dropout(rate)
            self.ffn = keras.Sequential([keras.layers.Dense(ff_dim, activation="relu"),
                                         keras.layers.Dense(embed_dim)])
            self.dropout = keras.layers.Dropout(rate)
            self.layernorm = keras.layers.LayerNormalization()

        def call(self, inputs):
            positions = keras.ops.arange(start=0, stop=keras.ops.shape(inputs)[-1],
step=1)
            x = self.token_emb(inputs) + self.pos_emb(positions)
            attn = self.self_att(x, x)
            attn = self.self_dropout(attn)
            x = self.self_layernorm(x + attn)
            f = self.ffn(x)
            f = self.dropout(f)
            return self.layernorm(x + f)

    layer = encoder(vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate)
    return layer
```

Estructura de una capa personalizada. En Keras, una capa propia se define heredando de `keras.layers.Layer`. En `__init__` se **declaran** las sub-capas que la componen (se crean una sola vez, con sus pesos), y en `call` se **usan** para transformar la entrada. `super().__init__()` inicializa la clase base.

`__init__`, pieza por pieza:

- `token_emb`: tabla de 5000 × 128. Convierte cada identificador de palabra en un vector de 128 números.
- `pos_emb`: tabla de 26 × 128. Convierte cada posición en un vector de 128. Se usa `maxlen + 1` porque la entrada mide 26, no 25.

- `self_att`: atención multi-cabeza con 2 cabezas. `key_dim=embed_dim` fija la dimensión interna de cada cabeza en 128; `dropout=rate` aplica dropout a los pesos de atención.
- `self_layernorm` y `self_dropout`: normalización y dropout del bloque de atención.
- `ffn`: red feed-forward de dos capas densas, 128 → 32 (con ReLU) → 128. Procesa cada posición por separado y añade capacidad no lineal.
- `dropout` y `layernorm`: normalización y dropout del bloque feed-forward.

`call`, paso a paso:

1. `positions = arange(0, 26)` genera `[0, 1, 2, …, 25]`. Se usa `keras.ops.shape(inputs)[-1]` para leer la longitud de la secuencia dinámicamente.
2. `x = token_emb(inputs) + pos_emb(positions)`: el vector de cada token se suma con el vector de su posición. Resultado: `(lote, 26, 128)`.
3. `attn = self_att(x, x)`: auto-atención; cada posición mezcla información de todas las demás.
4. `self_dropout(attn)` y `self_layernorm(x + attn)`: dropout, conexión residual y normalización.
5. `f = ffn(x)`, `dropout(f)`, `layernorm(x + f)`: segundo sub-bloque con la misma estructura residual.

Es exactamente un bloque de encoder del Transformer original.

CELDA 17 Celda de prueba

CÓDIGO

```
# TEST_CELL
encoder_layer = get_EncoderLayer(5000, 25, 128, 2, 32, .1)
encoded_x = encoder_layer(train_en_vec[:10])
print(f'Codificación de sequencias de entrenamiento: {encoded_x.shape}')
```

SALIDA

```
Codificación de sequencias de entrenamiento: (10, 26, 128)
```

Se codifican las primeras 10 frases. Para cada una, el encoder produce 26 vectores de 128 valores: uno por posición, cada uno con información de toda la frase. Esto es el “contexto” que el decodificador va a consultar.

CELDAS 18–19 Test 3_1 y 3_2

El test 3_1 revisa la estructura de la capa: que existan los atributos esperados, que sean del tipo correcto y que tengan los parámetros correctos. El test 3_2 compara los valores numéricos de la salida con la referencia. **Resultado: 100 y 100.**

Qué hubo que corregir. La primera versión usaba nombres propios para los atributos (`att`, `norm1`, `drop1`, …) y daba 0 puntos con el error *“'encoder' object has no attribute 'self_att'”*. Para averiguar qué esperaba el grader se usó una celda de diagnóstico temporal que registraba cada atributo que el grader intentaba leer. Resultaron ser: `token_emb`, `pos_emb`, `self_att`, `self_layernorm`, `self_dropout`, `ffn`, `dropout` y `layernorm`. Además, el grader exigía que la atención tuviera `dropout=rate` (*“se esperaba 0.1, pero se obtuvo 0.0”*). Con los nombres y el dropout corregidos, y con `pos_emb` de 26 posiciones, los valores numéricos coincidieron y el conteo de parámetros cuadró en 784 160.

5f. Punto 4 – Decodificador

> **Qué se pedía.** Una capa personalizada que represente el decodificador de un Transformer, con los mismos parámetros que el codificador. Su método `call` recibe dos entradas: los tokens de emojis (`inputs`) y la salida del codificador (`context_tokens`).

CELDA 20 Función calificada `get_DecoderLayer`

CÓDIGO

```
# FUNCIÓN CALIFICADA get_DecoderLayer
def get_DecoderLayer(vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate):

    class Decoder(keras.layers.Layer):
        def __init__(self, vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate=0.1):
            super().__init__()
            self.token_emb = keras.layers.Embedding(input_dim=vocab_size,
output_dim=embed_dim)
            self.pos_emb = keras.layers.Embedding(input_dim=maxlen,
output_dim=embed_dim)
            self.self_att = keras.layers.MultiHeadAttention(num_heads=num_heads,
key_dim=embed_dim, dropout=rate)
            self.self_layernorm = keras.layers.LayerNormalization()
            self.self_dropout = keras.layers.Dropout(rate)
            self.cross_att = keras.layers.MultiHeadAttention(num_heads=num_heads,
key_dim=embed_dim, dropout=rate)
            self.cross_layernorm = keras.layers.LayerNormalization()
            self.cross_dropout = keras.layers.Dropout(rate)
            self.ffn = keras.Sequential([keras.layers.Dense(ff_dim, activation="relu"),
                                         keras.layers.Dense(embed_dim)])
            self.dropout = keras.layers.Dropout(rate)
            self.layernorm = keras.layers.LayerNormalization()
            self.out_dropout = keras.layers.Dropout(rate)
            self.out_layer = keras.layers.Dense(vocab_size, activation="softmax")

        def call(self, inputs, context_tokens):
            positions = keras.ops.arange(start=0, stop=keras.ops.shape(inputs)[-1],
step=1)
            x = self.token_emb(inputs) + self.pos_emb(positions)
            attn = self.self_att(x, x, use_causal_mask=True)   # auto-atención causal
            attn = self.self_dropout(attn)
            x = self.self_layernorm(x + attn)
            attn = self.cross_att(x, context_tokens)           # atención cruzada con el
encoder
            attn = self.cross_dropout(attn)
            x = self.cross_layernorm(x + attn)
            f = self.ffn(x)
            f = self.dropout(f)
            x = self.layernorm(x + f)
            x = self.out_dropout(x)
            return self.out_layer(x)

    layer = Decoder(vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate)
    return layer
```

Diferencias respecto al codificador:

1. `pos_emb` tiene **25** posiciones (`maxlen`, sin el +1) porque la entrada del decodificador son los emojis sin el último token.
2. `self_att(x, x, use_causal_mask=True)`: la auto-atención lleva **máscara causal**. La posición *i* solo puede mirar las posiciones 0 … *i*. Sin esto, al entrenar, el modelo vería el emoji que debe predecir y "haría trampa"; luego, al generar de verdad, fallaría porque ese futuro no existe.
3. `cross_att(x, context_tokens)`: el bloque de **atención cruzada**. Las consultas vienen de los emojis y las claves y valores vienen del codificador. Aquí es donde el decodificador "lee" la frase en inglés: cada posición de emojis decide a qué palabras del texto prestar atención. Es el puente entre las dos mitades del modelo.
4. `out_layer = Dense(5000, activation="softmax")`: convierte cada vector de 128 en 5000 probabilidades, una por cada token del vocabulario de emojis. Se usa softmax (y no logits crudos) porque la función de pérdida del punto 6 está definida con `from_logits=False`. `out_dropout` aplica dropout justo antes.

El flujo de `call` es: embeddings → auto-atención causal (residual + norm) → atención cruzada (residual + norm) → feed-forward (residual + norm) → dropout → softmax.

CELDA 21 Celda de prueba

CÓDIGO

```
# TEST_CELL
decoder_layer = get_DecoderLayer(5000, 25, 128, 2, 32, .1)
decoded_x = decoder_layer(train_emoji_vec[:10,:-1],encoded_x)
print(f'Decodificación de sequencias de entrenamiento: {decoded_x.shape}')
```

SALIDA

```
Decodificación de sequencias de entrenamiento: (10, 25, 5000)
```

Se pasan los emojis de 10 frases sin su último token (`[:10, :-1]`, forma 10 × 25) junto con la salida del codificador. El resultado es 10 × 25 × 5000: para cada frase y cada posición, una distribución de probabilidad sobre los 5000 tokens posibles.

CELDAS 22–23 Test 4_1 y 4_2

Resultado: 100 y 100.

Qué hubo que corregir. Lo mismo que en el codificador: nombres de atributos exactos (se añaden `cross_att`, `cross_layernorm`, `cross_dropout`, `out_dropout` y `out_layer`) y `dropout=rate` en las dos capas de atención. El test 4_1 informaba explícitamente *"El valor de dropout no es el esperado en self_att. Se esperaba 0.1, pero se obtuvo 0.0"*.

5g. Punto 5 – Construir el modelo

> **Qué se pedía.** Implementar `get_model`, que construye el Transformer completo combinando codificador y decodificador, y devuelve un `keras.Model`.

CELDA 24 Función calificada `get_model`

CÓDIGO

```
# FUNCIÓN CALIFICADA get_model
def get_model(max_len, vocab_size, embed_dim, num_heads, ff_dim, rate):
    # Fijamos una semilla para efectos de reproducibiidad
    np.random.seed(0)
    keras.utils.set_random_seed(0)

    keras.backend.clear_session()

    encoder_inputs = keras.Input(shape=(max_len + 1,))
    decoder_inputs = keras.Input(shape=(max_len,))
    enc = get_EncoderLayer(vocab_size, max_len, embed_dim, num_heads, ff_dim, rate)
    dec = get_DecoderLayer(vocab_size, max_len, embed_dim, num_heads, ff_dim, rate)
    context = enc(encoder_inputs)
    outputs = dec(decoder_inputs, context)
    model = keras.Model([encoder_inputs, decoder_inputs], outputs)
    return model
```

- Las semillas y `clear_session()` (venían en la plantilla) reinician el estado de Keras y el azar, para que el modelo se construya idéntico cada vez.
- `keras.Input(shape=(26,))` y `keras.Input(shape=(25,))` declaran las dos entradas simbólicas: el texto en inglés y los emojis desplazados. El tamaño del lote no se declara (aparece como `None` en el resumen) porque puede variar.
- Se instancian las capas de los puntos 3 y 4 con los mismos hiperparámetros.
- `context = enc(encoder_inputs)`: el inglés entra al codificador. `outputs = dec(decoder_inputs, context)`: los emojis y el contexto entran al decodificador.
- `keras.Model([entradas], salida)` arma el grafo completo usando la API funcional de Keras.

CELDA 25 Celda de prueba con `model.summary()`

CÓDIGO

```
# TEST_CELL
max_len = 25
vocab_size = 5000
embed_dim = 128
num_heads = 2
ff_dim = 32
rate = 0.01

model = get_model(max_len, vocab_size, embed_dim, num_heads, ff_dim, rate)

model.summary()
```

Capa (tipo)	Forma de salida	Parámetros	Conectada a
input_layer (InputLayer)	(None, 26)	0	–
input_layer_1 (InputLayer)	(None, 25)	0	–
encoder (encoder)	(None, 26, 128)	784 160	input_layer
decoder (Decoder)	(None, 25, 5000)	1 561 256	input_layer_1, encoder
Total de parámetros (entrenables)		**2 345 416 (8,95 MB)**	

De dónde salen los números. Los 784 160 del codificador son: 640 000 de `token_emb` (5000 × 128) + 3 328 de `pos_emb` (26 × 128) + 131 968 de la atención (proyecciones Q, K, V y de salida) + 8 352 de la `ffn` + 512 de las dos normalizaciones. Los 1 561 256 del decodificador: 640 000 + 3 200 (25 × 128) + 263 936 de las dos atenciones + 8 352 + 768 de las tres normalizaciones + 645 000 de `out_layer` (128 × 5000 + 5000). Esa última capa es la más grande del modelo porque debe producir 5000 salidas. La coincidencia exacta con el resumen esperado confirma que la arquitectura es la correcta.

CELDAS 26–27 **Test 5_1 y 5_2**

Resultado: 100 y 100.

5h. Punto 6 – Entrenamiento

CELDA 28 Función de pérdida `causal_loss` (venía dada)

CÓDIGO

```
def causal_loss(y_true,y_pred):
    masc = keras.ops.cast(y_true != 0, dtype="float32")
    y_true = keras.ops.one_hot(y_true, 5000, axis=-1)

    loss_fn = keras.losses.CategoricalCrossentropy(from_logits=False, reduction='none')
    loss = loss_fn(y_true, y_pred)
    loss = loss * masc
    loss = keras.ops.sum(loss)/keras.ops.sum(masc)
    return loss
```

- `masc`: vale 1 donde hay un token real y 0 donde hay relleno. `cast` lo convierte a número decimal.
- `one_hot(y_true, 5000)`: el identificador `7` se convierte en un vector de 5000 ceros con un 1 en la posición 7, el formato que espera la entropía cruzada categórica.
- `CategoricalCrossentropy(from_logits=False, reduction='none')`: mide la distancia entre la probabilidad predicha y la real. `from_logits=False` indica que la salida del modelo ya es softmax. `reduction='none'` devuelve una pérdida por posición en lugar de promediarla.
- `loss * masc` anula la pérdida en las posiciones de relleno, y la división final promedia solo sobre las posiciones reales. Así los ceros de padding no penalizan al modelo.

Qué se pedía. Completar `train_model`, que recibe el modelo, los cuatro arreglos vectorizados, la tasa de aprendizaje, las épocas y el tamaño de lote, y devuelve el objeto `history`. Nota del enunciado: “recuerde desfasar por un token la salida esperada de la entrada”.

CELDA 29 Función calificada `train_model`

CÓDIGO

```
# FUNCIÓN CALIFICADA train_model
def train_model (model, train_text, train_emoji, test_text, test_emoji, learning_rate,
epochs, batch_size):
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
                  loss=causal_loss, metrics=['accuracy'])
    # El decodificador recibe la secuencia sin el último token
    # y debe predecir la secuencia desfasada un token
    history = model.fit(
        [train_text, train_emoji[:, :-1]], train_emoji[:, 1:],
        validation_data=([test_text, test_emoji[:, :-1]], test_emoji[:, 1:]),
        epochs=epochs, batch_size=batch_size)

    return history
```

- `model.compile(...)` configura el entrenamiento: el optimizador **Adam** (ajusta los pesos siguiendo el gradiente, con la tasa de aprendizaje indicada), la función de pérdida `causal_loss` y la métrica `accuracy` para reportar.
- `model.fit(...)` entrena. El primer argumento es la lista de las dos entradas; el segundo, el objetivo. `validation_data` evalúa en prueba al final de cada época. `epochs` es cuántas veces se

recorre todo el conjunto y `batch_size` cuántas frases se procesan por paso.

- `train_emoji[:, :-1]`: todas las filas, todas las columnas **menos la última** (25 tokens). Es la entrada del decodificador.
- `train_emoji[:, 1:]`: todas las filas, todas las columnas **menos la primera** (25 tokens). Es lo que debe predecir.

El desfase (teacher forcing). Para la secuencia `[BOS] ⚖️ ❓ [EOS]`:

Posición	Entrada al decoder `[:, :-1]`	Objetivo `[:, 1:]`
0	[BOS]	⚖️
1	⚖️	❓
2	❓	[EOS]

En cada posición el modelo ve el token actual (y los anteriores, gracias a la máscara causal) y debe predecir el siguiente. Durante el entrenamiento se le da la secuencia correcta como entrada; en la generación real usará sus propias predicciones. Al quitar un token por cada extremo, las secuencias de 26 quedan en 25, que es la longitud que espera el decodificador.

CELDA 30 Llamada al entrenamiento

CÓDIGO

```
history = train_model(model, train_en_vec, train_emoji_vec, test_en_vec, test_emoji_vec,
1e-4, 40, 256)
```

40 épocas, tasa de aprendizaje 0,0001 y lotes de 256 frases. Con 6605 frases salen 26 lotes por época. Salida resumida (el log completo tiene 40 líneas):

SALIDA

```
Epoch 1/40
26/26 ━━━━━━━━━━━━━━━━━━━━ 38s 755ms/step - accuracy: 0.0256 - loss: 8.0254 -
val_accuracy: 0.0400 - val_loss: 7.2922
Epoch 10/40
26/26 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step  - accuracy: 0.0400 - loss: 5.2860 -
val_accuracy: 0.0400 - val_loss: 3.8865
Epoch 19/40
26/26 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step  - accuracy: 0.0400 - loss: 4.2978 -
val_accuracy: 0.0533 - val_loss: 3.1100
Epoch 26/40
26/26 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step  - accuracy: 0.1319 - loss: 4.0246 -
val_accuracy: 0.0864 - val_loss: 3.7626
Epoch 40/40
26/26 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step  - accuracy: 0.0502 - loss: 3.4791 -
val_accuracy: 0.0470 - val_loss: 3.8522
```

La interpretación detallada de estos números está en la sección 7.

CELDA 32 Test 6_1

Resultado: 100. Este test califica `train_model`; no compara los valores del log, que dependen del hardware y de la versión de Keras.

5i. Generación de ejemplo

CELDA 31 Decodificación autorregresiva (venía dada)

CÓDIGO

```
current_token = 3
text_encoding = np.zeros([1,25])
text_encoding[0,0] = current_token

for i in range(24):
    token_prediction = model([np.expand_dims(test_en_vec[8], axis=0), text_encoding])
    current_token = np.argmax(token_prediction[0,i])
    text_encoding[0,i+1] = int(current_token)
    if current_token == 2:
        break

emoji_vocabulary = np.array(emoji_vectorizer.get_vocabulary())
en_vocabulary = np.array(en_vectorizer.get_vocabulary())
print(' '.join(en_vocabulary[list(test_en_vec[8].astype(int))]))
print(' '.join(emoji_vocabulary[list(text_encoding[0].astype(int))]))
```

- `current_token = 3`: se arranca con `[BOS]`. `text_encoding` es un arreglo de 1 × 25 lleno de ceros con el 3 en la primera posición.
- `test_en_vec[8]` es una frase del conjunto de **prueba**; el modelo nunca la vio. `np.expand_dims(..., axis=0)` le añade la dimensión de lote (1 × 26) que el modelo exige.
- En cada vuelta del bucle se pasa `[frase, emojis hasta ahora]` al modelo, se mira la posición *i* de la salida (5000 probabilidades), se toma la más alta con `argmax` y se escribe en la posición *i+1*.
- Si el token elegido es `2` (`[EOS]`) el bucle se detiene; si no, continúa hasta 24 pasos.
- Al final, `get_vocabulary()` devuelve la lista de tokens en el orden de sus identificadores, y se usa para traducir los números de vuelta a palabras y emojis.

SALIDA

```
[BOS] we also have a common responsibility to ensure that this takes place with legal
certainty while [UNK] fundamental rights [EOS]
[BOS] 🇪🇺❌🚫 [EOS]
```

El `[UNK]` del inglés corresponde a una palabra que no estaba entre las 5000 más frecuentes del entrenamiento. El modelo generó la cadena 🇪🇺❌🚫 y después `[EOS]`: eligió un token del vocabulario de emojis y supo terminar. Se analiza en la sección 7.

5j. Evaluación y envío

CELDA 33 `grader.submit_task(globals())`

Envía todas las funciones a UNCode como entrega oficial. Es el único paso que no se ejecutó durante el trabajo, porque constituye el envío formal de la tarea y esa decisión corresponde a la estudiante. Antes de ejecutarla conviene volver a correr la celda de inicio de sesión y verificar que todos los tests sigan en verde.

6. El hilo conductor: una frase de principio a fin

Para fijar la secuencia completa, se sigue la fila 0 de los datos a través de cada etapa.

Etapa	Texto en inglés	Emojis
Dato crudo	The ratification and implementation of the updated ILO conventions vote	✅🗳️
Punto 1 – preprocess	[BOS] the ratification and implementation of the updated ilo conventions vote [EOS]	[BOS] ✅🗳️ [EOS]
Punto 2 – vectorize	[3, id(the), id(ratification), …, id(vote), 2, 0, 0, …] (26 enteros)	[3, id(✅🗳️), 2, 0, 0, …] (26 enteros)
Punto 3 – encoder	26 vectores de 128 números, cada uno con contexto de toda la frase	–
Punto 6 – desfase	–	Entrada: [3, id(✅🗳️), 2, 0, …] (25) Objetivo: [id(✅🗳️), 2, 0, 0, …] (25)
Punto 4 – decoder	Para cada una de las 25 posiciones, 5000 probabilidades. En la posición 0 (ve [BOS] y la frase) debe dar la mayor probabilidad a ✅🗳️; en la posición 1 (ve ✅🗳️) debe dar la mayor a [EOS].	
Punto 6 – pérdida	`causal_loss` mide el error solo en las posiciones 0 y 1 (las demás son relleno) y Adam ajusta los 2,3 millones de parámetros para reducirlo.	
Generación	Con una frase nueva: [BOS] → predice un token → lo añade → predice el siguiente → … → [EOS].	

7. Resultados y análisis

7.1 Calificación de los tests

Test	Qué evalúa	Primera versión	Versión final
1_1, 1_2	`preprocess`	80, 80	100, 100
2_1, 2_2	`vectorize`	100, 100	100, 100
3_1, 3_2	`get_EncoderLayer` (estructura, valores)	0, 10	100, 100
4_1, 4_2	`get_DecoderLayer` (estructura, valores)	0, 100	100, 100
5_1, 5_2	`get_model`	100, 100	100, 100
6_1	`train_model`	100	100

Las once pruebas del calificador quedaron en 100.

7.2 Evolución del entrenamiento

Época	loss (entrenamiento)	accuracy	val_loss (prueba)	val_accuracy
1	8,03	0,026	7,29	0,040
5	6,37	0,040	5,51	0,040
10	5,29	0,040	3,89	0,040
15	4,59	0,040	3,36	0,040
19	4,30	0,040	**3,11**	0,053
26	4,02	0,132	3,76	0,086
30	3,84	0,066	3,65	0,055
40	3,48	0,050	3,85	0,047

El modelo aprendió. La pérdida de entrenamiento bajó de 8,03 a 3,48. Como referencia, adivinar al azar entre 5000 tokens daría una pérdida cercana a ln(5000) ≈ 8,5; el valor inicial está justo ahí, y la caída sostenida muestra que el modelo fue captando regularidades entre el texto y los emojis.

Hay sobreajuste a partir de la época 19. La pérdida de validación bajó hasta 3,11 y después subió y se estabilizó alrededor de 3,8, mientras la de entrenamiento seguía bajando. Ese cruce es la señal clásica de que el modelo empezó a memorizar frases de entrenamiento en lugar de generalizar. Si se buscara el mejor modelo posible, lo adecuado sería detener el entrenamiento cerca de la época 19 (*early stopping*) o aumentar la regularización. Para la tarea, que solo exige que `train_model` funcione, no era necesario.

Por qué `accuracy` se queda en 0,04. Ese valor es exactamente 1/25. De las 25 posiciones de salida, el modelo acierta una: casi con seguridad el `[EOS]` de la posición 1, que es fácil de predecir porque casi todas las secuencias de emojis tienen un solo token real. Las 23 posiciones de relleno no se entrenan (la pérdida las enmascara), de modo que allí nunca acierta, y eso arrastra la métrica hacia abajo. Por tanto `accuracy` no es un buen indicador en este problema; `val_loss` sí lo es.

Por qué la tarea es difícil. Como cada cadena de emojis es un solo token, el modelo debe elegir la cadena exacta (por ejemplo ✅🗳️) entre miles de combinaciones distintas que aparecen pocas veces cada una. Predecir ✅ y luego 🗳️ por separado sería mucho más fácil, pero no es lo que exige el calificador. Con 6605 ejemplos, un vocabulario de salida tan fragmentado y un modelo pequeño, una `val_loss` de 3,1–3,8 es un resultado razonable.

Comparación con la salida esperada del notebook. El enunciado muestra una pérdida de entrenamiento de 0,12 y una de validación de 4,86 en la época 40. Esa diferencia tan grande entre entrenamiento y validación indica un sobreajuste mucho mayor que el obtenido aquí; los valores absolutos cambian con la versión de Keras, el hardware y detalles de inicialización, así que no son comparables uno a uno. El test 6_1 no los evalúa.

7.3 El ejemplo de generación

Para la frase de prueba *“we also have a common responsibility to ensure that this takes place with legal certainty while [UNK] fundamental rights”*, el modelo generó 🇪🇺❌🚫 y luego `[EOS]`.

- Mecánicamente, el proceso funcionó: el modelo eligió un token del vocabulario de emojis y supo terminar la secuencia.
- Semánticamente, el resultado es parcial. La bandera de la Unión Europea encaja con el corpus (intervenciones del Parlamento Europeo), pero ❌🚫 no refleja bien "certeza legal" ni "derechos fundamentales". Es coherente con una pérdida de validación cercana a 3,8: el modelo captó patrones generales del dominio, no el matiz de cada frase.

8. Lo que hubo que corregir para obtener el 100 %

Punto	Problema detectado	Diagnóstico	Solución
1	80/100: "El resultado de text_out no es el esperado"	Se probaron cuatro variantes de preprocesamiento contra el grader	No separar los emojis; solo añadir [BOS] y [EOS]
3 y 4	0/100: "'encoder' object has no attribute 'self_att'"	Celda de diagnóstico que registró cada atributo leído por el grader	Renombrar los atributos: `self_att`, `self_layernorm`, `self_dropout`, `cross_att`, `cross_layernorm`, `cross_dropout`, `ffn`, `dropout`, `layernorm`, `out_dropout`, `out_layer`
3 y 4	"El valor de dropout no es el esperado en self_att. Se esperaba 0.1, pero se obtuvo 0.0"	Mensaje explícito del test	`MultiHeadAttention(..., dropout=rate)` en todas las atenciones
2, 5, 6	Ninguno	–	Pasaron a la primera

La lección metodológica es que un calificador automático puede evaluar no solo el comportamiento de una función sino su estructura interna. Cuando un test falla con un mensaje poco informativo, instrumentar el código para observar qué inspecciona el grader es más eficiente que adivinar.

9. Conclusiones

- La tarea pedía construir, desde sus piezas, un Transformer encoder-decoder para traducir texto en inglés a emojis, y entrenarlo con el esquema estándar de desfase de un token.
- El notebook sigue el flujo clásico de un problema seq2seq: limpieza del texto, vectorización, codificador, decodificador, ensamblaje, entrenamiento y generación autorregresiva. Cada punto calificado corresponde a una de esas etapas.
- Las seis funciones quedaron implementadas y las once pruebas del calificador obtuvieron 100.
- El modelo entrenado aprendió regularidades reales (la pérdida cayó de 8,0 a 3,5) y empezó a sobreajustar a partir de la época 19, lo que es esperable con un conjunto de 6605 ejemplos y un vocabulario de salida en el que cada cadena de emojis es un token.
- El resultado de generación es mecánicamente correcto y semánticamente parcial, en línea con la capacidad del modelo y los datos disponibles. Mejoras naturales serían detener el entrenamiento

en la época de mejor validación, separar los emojis en tokens individuales (si el calificador lo permitiera) y usar más datos o un modelo con más bloques.

Créditos del notebook original: Profesor Fabio Augusto González; asistente docente Juan Sebastián Malagón Torres; coordinador de virtualización Edder Hernández Forero. Universidad Nacional de Colombia – Facultad de Ingeniería.